

Documentación del Proyecto

Portal Web de Tecnología — MTT

Índice

1. [Introducción](#)
 2. [Especificaciones del Proyecto](#)
 3. [Estructura del Proyecto](#)
 4. [Flujo del Usuario](#)
 5. [Sistema de Login y Registro](#)
 6. [Base de Datos](#)
 7. [Interfaz de Usuario](#)
 8. [Consideraciones de Seguridad](#)
-

1. Introducción

Campo	Dato
Nombre y Apellido del alumno	
Comisión	
Nombre y Apellido del Profesor	
Nombre del proyecto	

Sección pendiente de completar con los datos institucionales del alumno.

2. Especificaciones del Proyecto

El proyecto consiste en un **portal web de noticias e interacción sobre tecnología** (blog). Su objetivo principal es ayudar a personas no técnicas con interés en tecnología a mantenerse al día con las noticias del sector de forma simple y accesible.

A nivel funcional, el sistema opera como una comunidad: permite la lectura de artículos redactados por el equipo editorial, pero también les da a los lectores registrados herramientas para comentar, interactuar, personalizar su interfaz e incluso generar su propio contenido bajo ciertos permisos.

2.1 Funcionalidades principales del portal web

- **Navegación y lectura:** Acceso completo al catálogo de artículos publicados.
- **Página principal inteligente:** Bloque de noticias destacadas (Top 5 por cantidad de "me gusta") y feed de noticias recientes.
- **Búsqueda y filtros:** Consultas por palabras clave, categorías temáticas y etiquetas (tags).
- **Cuentas de usuario:** Registro, inicio y cierre de sesión, y eliminación de la propia cuenta.
- **Perfiles públicos:** Avatar, nombre, biografía, ubicación, educación y teléfono, con control de privacidad campo por campo.
- **Interacción social:**
 - Sistema de comentarios con paginación.
 - Reacción "me gusta" sobre publicaciones y comentarios.
 - Guardado de publicaciones favoritas (bookmarks) para lectura posterior.
 - Seguimiento entre usuarios (seguidores/seguídos).
- **Dashboard personal:** Panel con pestañas que agrupan los "me gusta", los comentarios propios, los destacados guardados y los blogs propios.
- **Módulo editorial:** Creación, edición y eliminación de publicaciones por parte de Editores y Administradores.
- **Gestión de taxonomía:** Administración de categorías y etiquetas.
- **Panel de administración:** Reportes de impacto, gestión de usuarios y editores, y configuración global del sitio.
- **Personalización de interfaz:** Modo claro/oscuro y colores de acento configurables según el rol o preferencia del usuario.

2.2 Tecnologías utilizadas

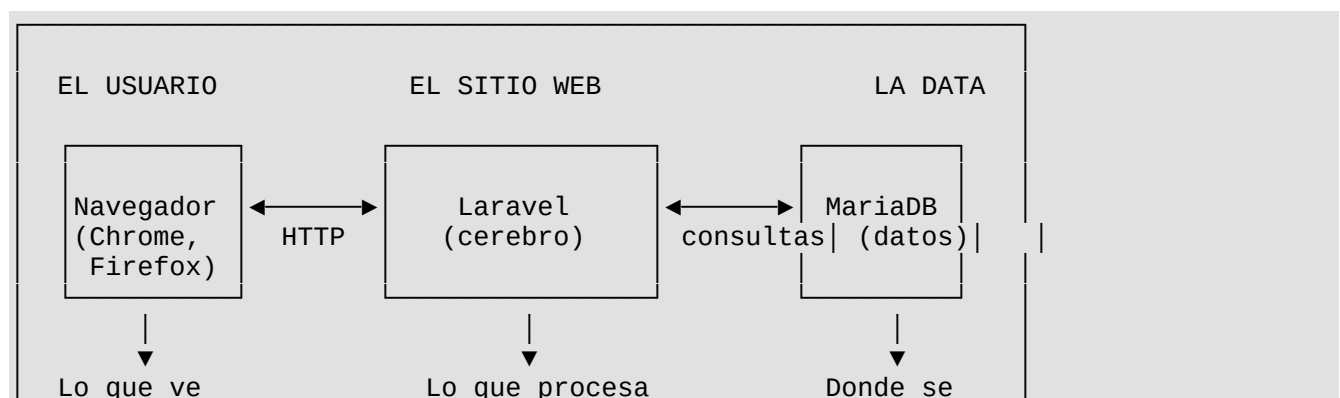
Para este proyecto se eligió un stack clásico pero sólido, priorizando la estabilidad del lado del servidor y la agilidad en la maquetación.

Tecnología	Entorno	Uso en el proyecto
PHP (v8.3+)	Servidor	Lenguaje base del backend: procesa la lógica de negocio, las peticiones HTTP y la comunicación con la base de datos.
Laravel (v13)	Servidor	Framework MVC que organiza rutas, controladores, modelos, plantillas, migraciones y módulos de seguridad.

Tecnología	Entorno	Uso en el proyecto
Blade	Servidor	Motor de plantillas de Laravel: renderiza HTML modular y reutilizable.
Eloquent ORM	Servidor	Mapeo objeto-relacional que simplifica las consultas a la base de datos mediante modelos PHP.
MariaDB	Servidor	Sistema gestor de base de datos relacional que almacena toda la información persistente.
JavaScript (vanilla)	Cliente	Comportamiento dinámico: alternar modo claro/oscuro (con <code>localStorage</code>) y toggles visuales de reacciones.
Bootstrap (v5.3.8)	Cliente	Framework CSS consumido por CDN para maquetar una interfaz limpia y responsiva.
SVG	Cliente	Biblioteca de 32 vectores para íconos reactivos (corazón, marcador, comentario) que adoptan el color de acento activo.
Composer	Herramienta	Gestor de dependencias de PHP.
Vite + npm	Herramienta	Build y empaquetado de assets del frontend.
Git	Herramienta	Control de versiones del código fuente.

Arquitectura del proyecto

El portal sigue el patrón de arquitectura **MVC (Modelo-Vista-Controlador)** distribuido entre servidor y cliente:



el usuario	las peticiones	guardan los datos
------------	----------------	-------------------

1. **Cliente (navegador):** muestra el HTML/CSS estilizado y ejecuta JavaScript para la interacción inmediata.
2. **Servidor (Laravel):** recibe las solicitudes, valida credenciales y permisos, y ejecuta la lógica de negocio.
3. **Persistencia (MariaDB):** guarda y entrega la información requerida por los modelos.

3. Estructura del Proyecto

3.1 Árbol de directorios y archivos principales

```

PP/
├── app/
│   ├── Enums/
│   │   ├── ColorAccent.php      # Colores de acento permitidos
│   │   └── Roles.php            # Roles del sistema (usuario/editor/admin)
│   ├── Http/
│   │   ├── Controllers/        # Controladores de cada sección
│   │   │   ├── AdminController.php # Reportes y configuración global
│   │   │   ├── AuthController.php  # Registro, login, logout
│   │   │   ├── CategoriaController.php # Categorías y etiquetas
│   │   │   ├── ComentarioController.php # Comentarios y sus likes
│   │   │   ├── DashboardController.php # Paneles personales
│   │   │   └── PublicacionController.php # Publicaciones, búsqueda, likes,
│   │   ├── UsuarioController.php # Perfiles, seguimiento y edición
│   │   ├── Middleware/
│   │   │   └── RoleMiddleware.php  # Control de acceso por roles
│   │   ├── Models/              # Modelos Eloquent (9 modelos)
│   │   ├── Providers/
│   │   │   └── AppServiceProvider.php # Paginación Bootstrap + variable $color
│   │   └── bootstrap/app.php      # Arranque de la app y alias del middleware
│   └── config/                  # Configuración de Laravel (auth, database,
etc.)
├── database/
│   ├── factories/              # Generación de datos sintéticos (Faker)
│   ├── migrations/             # Estructura e historial de tablas (15
migraciones)
│   ├── seeders/                # Poblado inicial de datos de prueba
│   └── specs/                  # Documentación del proyecto (este archivo,
DER, ML)
├── public/
│   ├── css/style.css           # Estilos propios (clases auxiliares)
│   ├── js/script.js            # Scripts del cliente (tema claro/oscuro,
toggles)
│   ├── build/                  # Assets compilados por Vite
│   └── storage/                # Enlace simbólico a storage/app/public
└── resources/

```

views/	# Plantillas Blade
layouts/	
template.blade.php	# Layout base maestro (navbar, alerts,
footer)	
components/	# Componentes UI reutilizables (cards, likes)
auth/	# Formularios de login y registro
publicacion/	# Vista, búsqueda, creación y edición de
artículos	
perfil/	# Vistas de perfil (ver, editar, crear
usuario)	
dashboard/	# Paneles personales por pestañas
categorias/	# Filtrado y gestión de taxonomía
admin/	# Panel admin (usuarios, editores, reportes)
routes/	
web.php	# Definición de todas las rutas del sitio
storage/app/public/	# Archivos subidos (pfps, portadas) y SVGs
tests/	# Tests de Laravel (unit/feature)
composer.json	# Dependencias de PHP
package.json	# Dependencias de Node/Vite

3.2 Propósito de los archivos y directorios principales

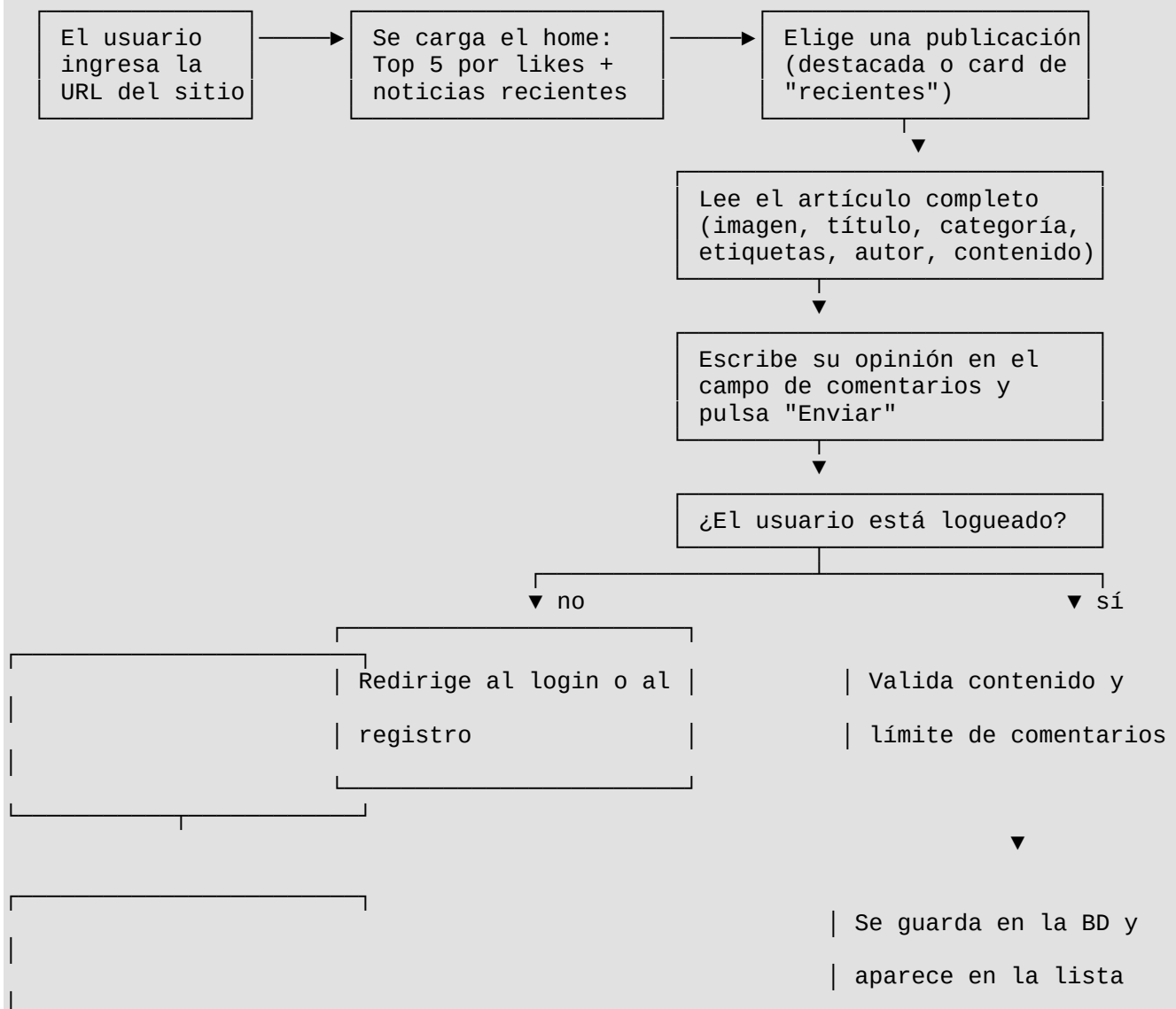
Ruta	Propósito
routes/web.php	Define los endpoints del sitio y agrupa las rutas protegidas por middleware (auth y rol).
app/Http/Middleware/RoleMiddleware.php	Verifica que el usuario autenticado posea alguno de los roles requeridos por la ruta.
app/Http/Controllers/*	Contienen la lógica de negocio de cada módulo: validan peticiones, consultan la base de datos y devuelven vistas o redirecciones.
app/Models/*	Modelos de Eloquent que mapean las tablas y definen las relaciones (pertenece a, tiene muchos, muchos a muchos).
app/Enums/*	Valores fijos del sistema: roles permitidos y colores de acento válidos.
app/Providers/AppServiceProvider.php	Configura la paginación con Bootstrap y comparte la variable \$color (accento activo) con todas las vistas.
database/migrations/*	Versionan la estructura de las tablas de la base de datos.
database/seeders/*	Pueblan la base de datos con información inicial (roles, configuraciones, usuarios, publicaciones y comentarios de prueba).
database/factories/*	Definen cómo generar datos sintéticos realistas para los seeders y tests.
resources/views/*	Plantillas Blade que conforman la interfaz de usuario.

Ruta	Propósito
public/js/script.js	Lógica de cliente: alternar tema claro/oscuro y mostrar/ocultar formularios de edición.
public/css/style.css	Estilos propios complementarios a Bootstrap.
storage/app/public/	Almacenamiento de imágenes subidas por los usuarios (avatares y portadas) y la biblioteca de SVGs.

4. Flujo del Usuario

4.1 Recorrido del usuario lector: desde el acceso hasta comentar

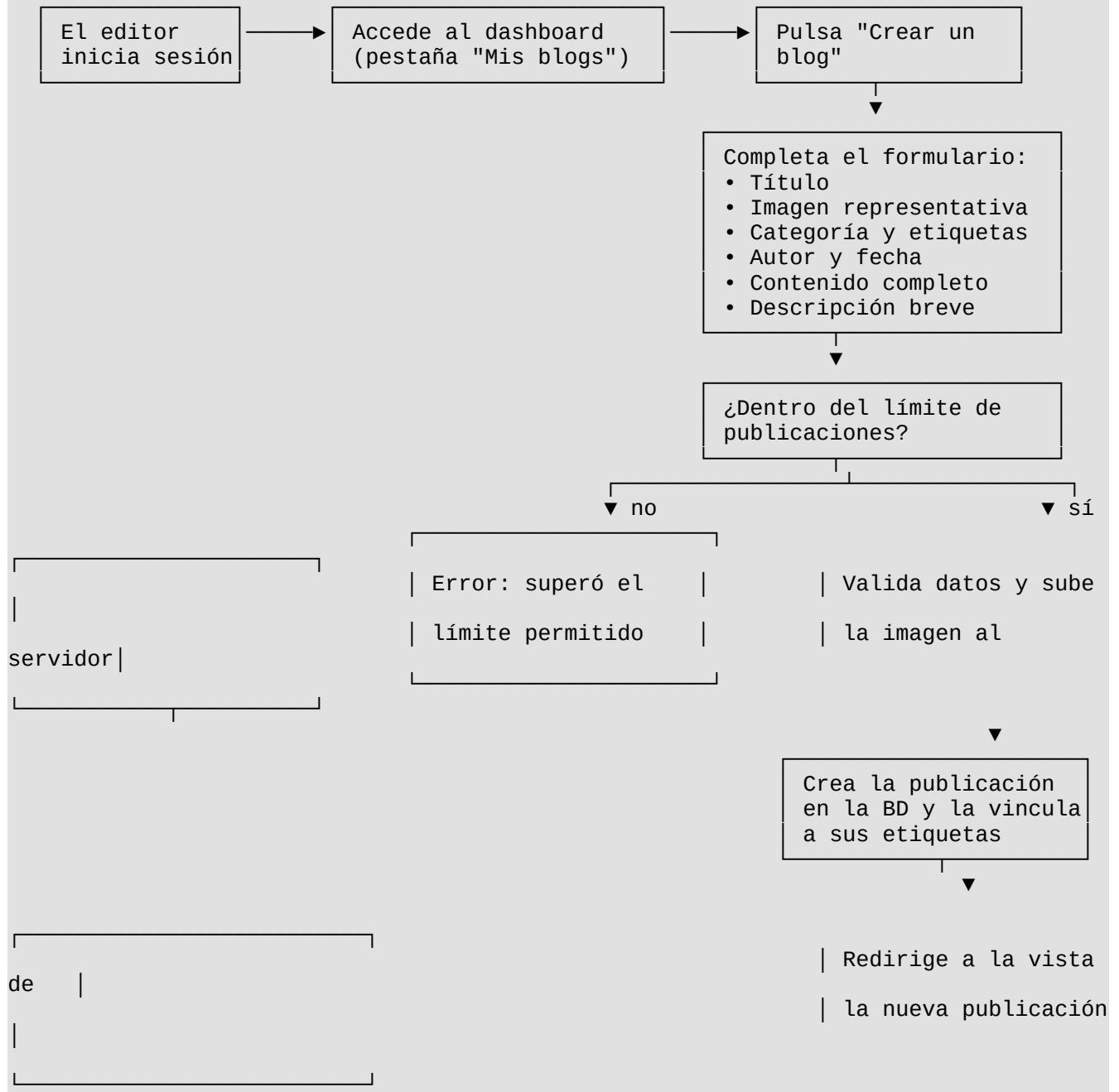
ACCESO AL PORTAL



Durante la lectura, el usuario también puede **reaccionar con "me gusta"**, **guardar** la publicación como favorita o **seguir al autor** desde su perfil, acciones que se alternan con un clic (toggle).

4.2 Flujo del editor: crear una publicación nueva

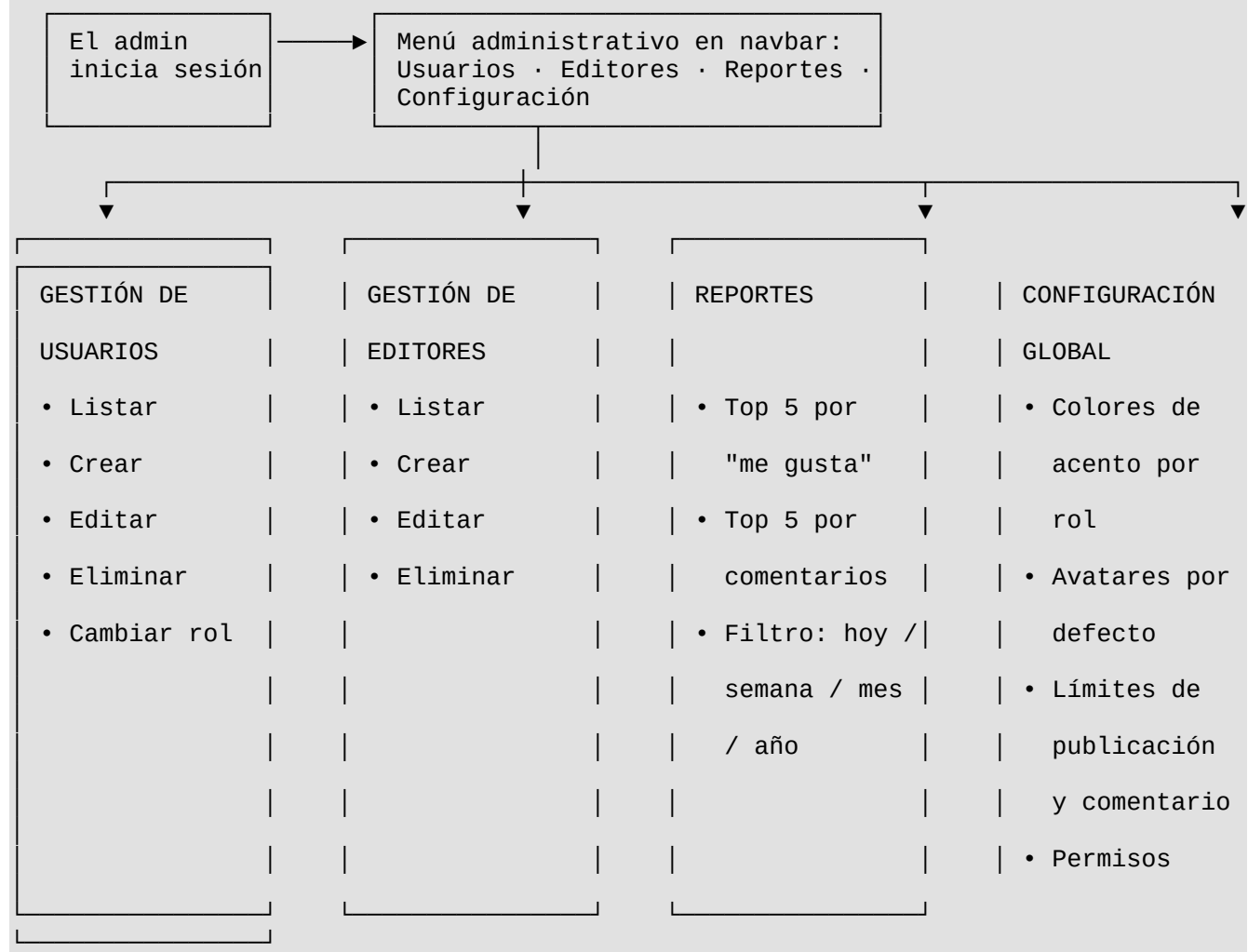
INICIO DE SESIÓN DEL EDITOR



El editor también puede **modificar** o **eliminar** sus propias publicaciones desde la vista del artículo o desde el dashboard, y **gestionar categorías y etiquetas**.

4.3 Flujo del administrador: gestión completa del blog

INICIO DE SESIÓN DEL ADMINISTRADOR



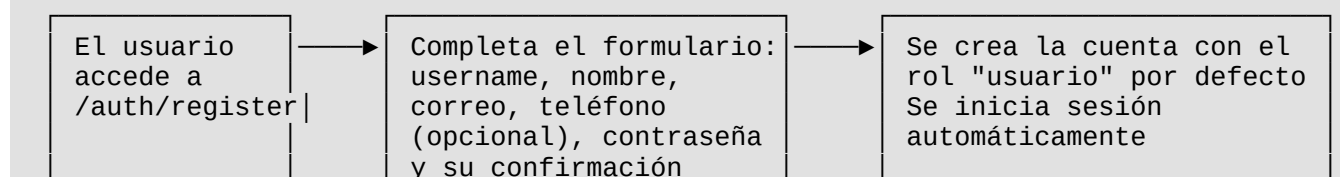
Además, el administrador puede **editar o eliminar cualquier publicación o comentario** del sitio, acceder al perfil de cualquier usuario y ver las estadísticas generales de la plataforma.

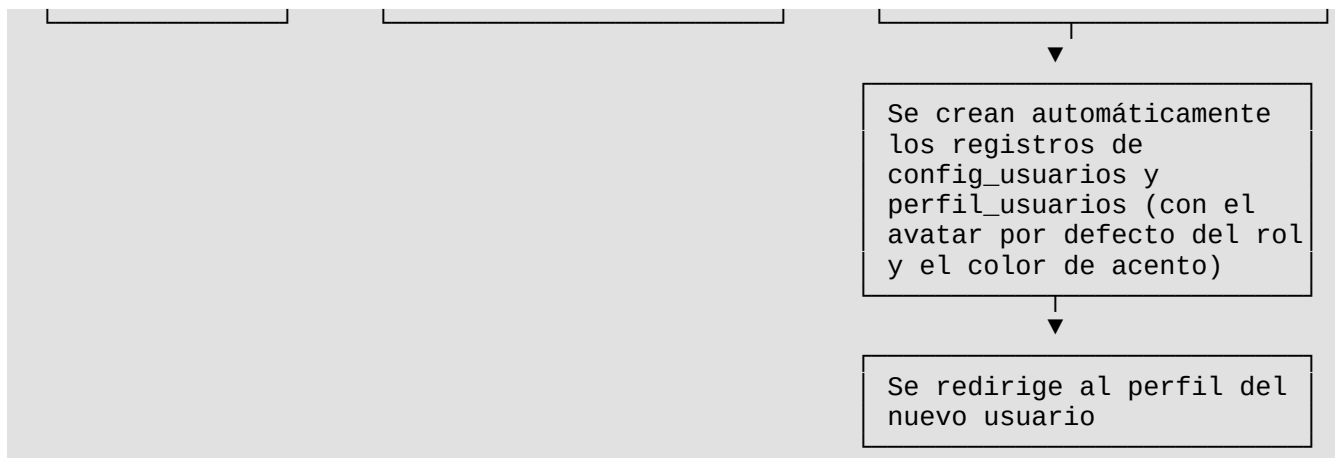
5. Sistema de Login y Registro

El portal implementa un **sistema de autenticación propio** (sin usar paquetes de terceros como Breeze o Jetstream), basado en sesiones y en el guard web de Laravel con el proveedor de usuarios Eloquent.

5.1 Registro de usuarios

REGISTRO:

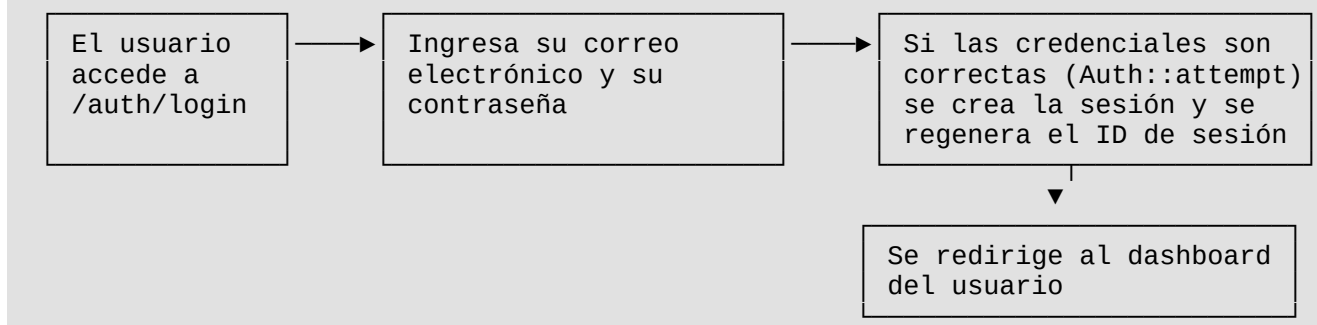




La contraseña se guarda **encriptada** mediante `Hash::make()` (bcrypt) y nunca se almacena en texto plano. Los campos `username` y `correo` son únicos y se validan contra la base de datos.

5.2 Inicio de sesión

INICIO DE SESIÓN:



Si las credenciales son incorrectas, se muestra el error *"El correo o la contraseña son incorrectos"* y se conserva el correo ingresado en el formulario (onlyInput).

5.3 Cierre de sesión y eliminación de cuenta

- **Cierre de sesión (/logout):** se ejecuta `Auth::logout()`, se **invalida la sesión** y se **regenera el token CSRF**, evitando el secuestro de sesión.
- **Eliminación de cuenta (/usuario/remover):** cierra la sesión, invalida la sesión y borra definitivamente al usuario autenticado de la base de datos. Las relaciones asociadas se eliminan en cascada.

5.4 Autorización y roles

El sistema define **tres roles** mediante el Enum Roles: `usuario`, `editor` y `admin`.

La autorización se implementa en dos capas:

1. **Middleware de rutas:** el middleware `RoleMiddleware` se registra con el alias `rol` en `bootstrap/app.php` y se aplica sobre grupos de rutas:

- `auth` → rutas privadas para cualquier usuario logueado (dashboard, comentarios, likes, edición de perfil).
- `auth + rol:admin,editor` → creación/edición/eliminación de publicaciones y gestión de categorías/etiquetas.
- `auth + rol:admin` → panel de administración (usuarios, editores, reportes y configuración).

2. **Chequeos en los controladores:** además del middleware, los controladores verifican permisos específicos, por ejemplo:

- Solo el **autor** o un **admin** pueden modificar/eliminar una publicación.
- Un usuario solo puede editar su **propio** comentario (si la configuración global lo permite) o eliminarlo.
- Un editor puede eliminar comentarios de terceros solo si el admin habilitó esa opción.
- Un usuario no puede editar el perfil de otro usuario (salvo el admin).

El método `hasRole(string $rol)` del modelo `Usuario` compara el rol asignado (a través de la relación `rol`) y devuelve `true` o `false`, y es la base de todas las comprobaciones de autorización.

5.5 Matriz de permisos por rol

Funcionalidad	Usuario	Editor	Administrador
Leer publicaciones, buscar y filtrar	✓	✓	✓
Comentar y reaccionar (likes)	✓	✓	✓
Guardar favoritos y seguir usuarios	✓	✓	✓
Gestionar su propio perfil	✓	✓	✓
Publicar y gestionar sus propias publicaciones	✗	✓	✓
Administrar categorías y etiquetas	✗	✓	✓
Editar/eliminar cualquier contenido	✗	✗	✓
Gestionar usuarios y editores	✗	✗	✓
Ver reportes y analíticas	✗	✗	✓
Cambiar la configuración global del	✗	✗	✓

Funcionalidad	Usuario	Editor	Administrador
sitio			

6. Base de Datos

6.1 Diagrama Entidad-Relación (DER)

Sección pendiente: insertar aquí el diagrama DER del proyecto (ver archivos `DER-PP.jpg` y `DER_PP.drawio` en `database/specs/`).

6.2 Modelo Lógico (ML)

Sección pendiente: insertar aquí el modelo lógico de la base de datos (ver archivo `ML_PP.xlsx` en `database/specs/`).

7. Interfaz de Usuario

7.1 Diseño general

La interfaz se construye sobre **Bootstrap 5.3.8** (cargado por CDN) y un pequeño archivo de estilos propio (`public/css/style.css`). Todas las páginas heredan de la plantilla base `layouts/template.blade.php`, que incluye:

- **Barra de navegación (navbar):** logo, enlaces de navegación (Home, Categorías, Publicaciones, y enlaces administrativos según el rol), buscador de palabras clave, menú de usuario (avatar, perfil, dashboard, cerrar sesión, eliminar cuenta) y el botón de alternancia claro/oscuro.
- **Alertas de feedback:** mensajes de éxito y errores de validación mostrados en la parte superior.
- **Footer:** créditos del proyecto.

El diseño es **responsivo**: utiliza el sistema de grillas de Bootstrap (`col-12`, `col-md-6`, `col-lg-4`, etc.) para adaptarse a pantallas móviles, tablets y escritorio.

7.2 Colores de acento

Todas las vistas reciben automáticamente la variable `$color` (inyectada por el `AppServiceProvider` mediante un `View::composer`). El color define la clase Bootstrap de acento (botones, enlaces, bordes) y depende de:

1. La preferencia personal del usuario (campo `color` en `config_usuarios`).

2. Si no la tiene, el color asignado a su rol en la tabla configuraciones (colorAccentoUsuario, colorAccentoEditor, colorAccentoAdmin).
3. Para visitantes, el color por defecto de los usuarios (colorAccentoUsuario).

Color de acento	Valor Bootstrap
Azul	primary
Verde	success
Rojo	danger
Aqua	info
Blanco	light
Negro	dark

7.3 Página principal (Home)

- **Top Noticias:** bloque con la publicación más popular en grande y las 4 siguientes en formato compacto (se ordenan por cantidad de "me gusta").
- **Noticias Recientes:** grilla de 9 tarjetas con las publicaciones más nuevas ordenadas por fecha.

7.4 Tarjetas de publicación

Cada publicación se renderiza con el componente reutilizable `components/card.blade.php` e incluye:

- Imagen de portada, título y descripción breve.
- Autor (enlazado a su perfil) y categoría (enlazada a su filtro).
- Contadores de "me gusta", guardados y comentarios.
- Fecha de publicación.
- Botón **Leer Más**, y botones **Modificar/Eliminar** visibles solo para el autor o un administrador.

7.5 Vista de artículo

Muestra la imagen en grande, título, categoría, etiquetas, autor, fecha y contadores de interacción. Debajo se presenta el contenido completo, un **formulario de comentarios** y la **lista de comentarios paginada** (10 por página), cada uno con su avatar, autor, contenido y reacción "me gusta".

7.6 Perfil de usuario

Incluye el avatar circular, nombre completo, @username, contadores de seguidores y seguidos, botón **Seguir/Dejar de seguir** (oculto en el propio perfil) y botón **Editar Perfil**. La información adicional (correo, ubicación, educación, teléfono) se muestra únicamente si el usuario la marcó como pública. Debajo aparece la biografía **"Sobre mí"** y los accesos al dashboard.

7.7 Dashboard personal

Panel organizado por **pestañas**:

- **Me gusta:** publicaciones a las que el usuario dio like.
- **Tus comentarios:** actividad del usuario en las conversaciones.
- **Tus destacados:** publicaciones guardadas como favoritas.
- **Mis blogs** (Editor/Admin): publicaciones propias con acciones de edición.
- **Blogs** (Admin): todas las publicaciones del sistema.

Cada pestaña muestra el total correspondiente y, para editores/administradores, el botón **Crear un blog**.

7.8 Panel de administración

- **Gestión de Usuarios / Editores:** tablas paginadas con avatar, nombre, username, correo, ubicación, educación y teléfono; botones para crear, modificar y eliminar cuentas.
- **Reportes:** dos rankings (Top 5 por "me gusta" y Top 5 por comentarios) filtrables por período (hoy, semana, mes, año).
- **Configuración:** formulario con tres bloques — apariencia (colores de acento y avatares por defecto por rol), permisos (interruptores booleanos) y límites (máximo de publicaciones y comentarios por usuario).

7.9 Tema claro/oscuro e íconos

- **Tema:** el botón del navbar alterna el atributo `data-bs-theme` del documento. La preferencia se guarda en `localStorage` (`theme` y `toggleIcon`), por lo que persiste entre páginas y sesiones.
 - **Íconos reactivos:** la biblioteca de **32 SVGs** (corazón, marcador y comentario en los 6 colores de acento, con y sin relleno, más los íconos de sol/luna) se sirve desde `storage/app/public/svg/`. El nombre del archivo se genera dinámicamente según el color de acento activo, garantizando coherencia visual en toda la interfaz.
-

8. Consideraciones de Seguridad

El proyecto implementa varias medidas para proteger los datos de los usuarios y la integridad del sistema:

8.1 Encriptación de contraseñas

- Todas las contraseñas se almacenan con **bcrypt** mediante `Hash::make()` (con 12 rondas, `BCRYPT_ROUNDS=12`).

- La verificación se realiza a través de `Auth::attempt()` / `Hash::check()`, nunca comparando cadenas en texto plano.
- El campo `password` del modelo `Usuario` está oculto en las serializaciones (`$hidden`), y en las tablas administrativas se muestra como `*****`.

8.2 Tokens CSRF

- Todos los formularios que modifican datos (`POST`, `PUT`, etc.) incluyen la directiva `@csrf` de Blade.
- Laravel valida automáticamente el token en cada petición de escritura, impidiendo ataques de falsificación de solicitudes entre sitios (CSRF).
- Al cerrar sesión o eliminar la cuenta, el token se **regenera** (`regenerateToken`) para invalidar peticiones previas.

8.3 Validación de entradas

- Cada controlador valida las entradas con las reglas de Laravel (requerido, tipos, longitudes máximas, formato de email, fechas, etc.) y con mensajes de error personalizados en español.
- **Unicidad:** `username` y `correo` son únicos en la base de datos (con `Rule::unique` que ignora al usuario en edición).
- **Enumerados:** roles y colores de acento se validan con `Rule::enum` para aceptar solo valores permitidos.
- **Existencia:** los identificadores foráneos se comprueban con la regla `exists:tabla,id`.

8.4 Validación de archivos subidos

- Solo se aceptan **imágenes** válidas (`mimes:image`) con un tamaño máximo de **5 MB**.
- Los archivos se renombran con un sello de tiempo único y se guardan en el almacenamiento público (`storage/app/public/pfps` o `publicaciones`), accesible mediante el enlace simbólico `storage:link`.
- El acceso público se sirve a través de Laravel, no por rutas directas de escritura.

8.5 Autorización basada en roles

- Las rutas privadas están protegidas por el middleware `auth` (requiere sesión iniciada).
- Las rutas administrativas y editoriales requieren además el middleware `rol` con los roles permitidos.
- Los controladores refuerzan la autorización con chequeos adicionales (autor de la publicación/comentario o rol administrador), evitando que un usuario modifique contenido que no le corresponde.

8.6 Manejo de sesiones

- Las sesiones se almacenan en la **base de datos** (tabla `sessions`), con datos como IP, user agent y actividad.
- Tras un inicio de sesión exitoso se regenera el ID de sesión (`regenerate()`), mitigando la **fijación de sesión**.
- Al cerrar sesión se invalida la sesión completa y se regenera el token.

8.7 Protección de datos personales

- Los datos del perfil (correo, ubicación, educación, teléfono) son **privados por defecto** y solo se muestran públicamente si el usuario activa el flag correspondiente.
- Incluso con el flag activado, un administrador siempre puede verlos; el resto de los visitantes solo ve lo permitido.
- La contraseña nunca se muestra ni se devuelve en las respuestas.

8.8 Escapado de salidas

- Las plantillas Blade usan la sintaxis `{{ }}` (escape automático de HTML) para mostrar datos dinámicos, previniendo ataques de **XSS (Cross-Site Scripting)**.
- Los valores de los formularios se rellenan con `old()` para reutilizar la entrada del usuario tras un error de validación sin reflejar contenido no escapado.

Documento elaborado para el proyecto "Portal Web de Tecnología — MTT", materia de Producción Web.